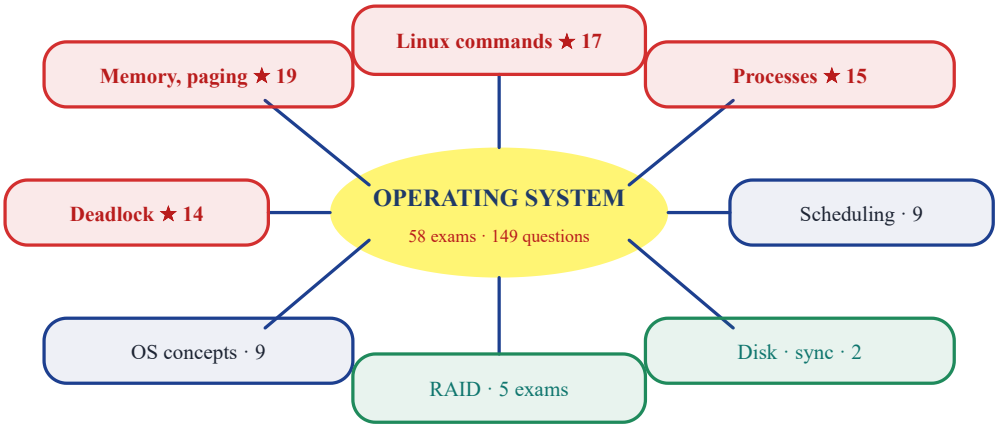
Operating System

every OS question asked in BCS · NTRCA · BPSC · bank · BUET-set papers, solved in 19 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
> ✎ **Asked in:** the exams it was taken from, e.g. BPSC AP 2023 · Milk Vita AM(CSE) 2023

No dated exam → the card names its source (NTRCA question bank, suggestion sheet). Green = worked result / output.

Most asked
① deadlock + 4 conditions
② paging vs segmentation
③ fragmentation, swapping
④ LRU page faults
⑤ Linux: chmod, ls -a, tail
⑥ PCB, process states
⑦ Gantt chart TAT / WT

Syllabus coverage

"OS: its role; concepts; structure; **Process**: model, IPC, **scheduling**; **Memory management**: swapping, paging, segmentation, virtual memory; I/O, disk; **Deadlock**: detection, prevention, recovery; File Systems". Linux commands and RAID are outside the line but asked in most BUET-set papers.

Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked answer / output

Contents

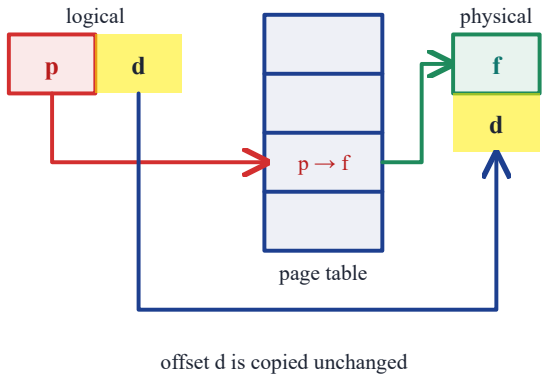
✓ Memory I–IV: paging, VM, LRU … p. 2–5
✓ Linux I–III: permissions, files … p. 5–8
✓ Processes I–III: states, PCB, threads … p. 8–10
✓ Synchronization …… p. 10
✓ Deadlock I–III, Banker's …… p. 11–13
✓ Scheduling I–III, Gantt charts … p. 13–15
✓ OS concepts, kernels …… p. 16–17
✓ RAID, disk and I/O …… p. 17–18
✓ Night before …… p. 18
✓ Practice MCQs + written …… p. 19

1 Paging vs segmentation

Q1: Difference between paging and segmentation.

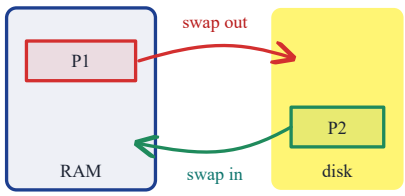
✎ **Asked in:** 17th NTRCA CSE 2023 · BTCL JAM 2024 · NTRCA question bank

Point	Paging	Segmentation
Unit	**fixed-size** pages / frames	**variable-size** segments
View	physical; invisible to the programmer	logical: code, data, stack
Address	(page no., offset)	(segment no., offset)
Table	page table → frame no.	segment table → base + limit
Waste	**internal** fragmentation	**external** fragmentation
Sharing	per page	per segment (natural)



Q2: What is swapping? Write the difference between internal and external fragmentation.

✎ **Asked in:** 17th NTRCA ICT 2023 · 18th NTRCA College ICT 2024 (Q8a) · "Explain swapping, virtual memory": Combined Bank SO(AME) 2021



Swapping: Moving a whole **process** (or its pages) out of RAM to a **backing store** on disk and bringing it back later, so more processes fit than RAM alone allows. Cost: disk I/O; too much of it → thrashing.

Point	Internal fragmentation	External fragmentation
Where	unused space **inside** an allocated block	free holes **between** blocks, each too small
Cause	fixed partitions / pages bigger than the need	variable partitions / segments come and go
Example	8 MB job in a 10 MB partition → 2 MB lost	3 holes of 4 MB, a 10 MB job cannot load
Remedy	smaller pages, best-fit partitions	**compaction**, paging

Memory trick: "**P-I, S-E**": Paging → Internal, Segmentation → External. Fixed size wastes inside, variable size outside.

2 Virtual memory and page faults

Q3: What do you mean by virtual memory? Why is it needed? Difference between physical and virtual memory, with its advantages and disadvantages.

✎ **Asked in:** BPSC AME 2021 · BPSC AP 2021 (need) · Combined Bank AME/AHE 2023 · Kernel, cache, VM and RAID: SPCBL AME 2021

Virtual memory gives each process a large, private logical address space; only the pages in use sit in RAM, the rest stay on disk and are loaded on demand (**demand paging**). **Needed** because programs are bigger than RAM, many programs run at once, and each needs protection from the others.

Physical memory	Virtual memory
real RAM chips	RAM + disk (swap) as one space
size = installed RAM	size = address width (2^{32} = 4 GB)
fast (ns)	a miss costs a disk access (ms)
addressed directly	MMU translates every address

Advantages: run programs larger than RAM · more processes at once · isolation and protection · easy sharing of libraries.

Disadvantages: slower on page faults · disk space used · page tables cost memory · thrashing if RAM is too small.

Q4: What is a page fault in computing systems? When does it occur?

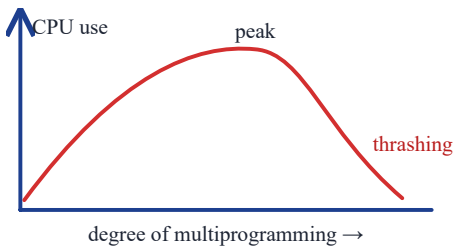
✎ **Asked in:** BCIC AP 2022 · "when does a page fault occur": NTRCA question bank

1. CPU issues an address; the MMU finds the page's **valid bit = 0 → trap** to the OS
2. OS checks the reference is legal (else: segmentation fault, kill)
3. finds a free frame, or chooses a **victim** by a replacement algorithm (writes it back if dirty)
4. reads the page from disk into the frame; updates the page table (valid = 1)
5. restarts the instruction that faulted

Q5: Define paging and thrashing in the context of OS.

✎ **Asked in:** BPSC AP 2023 (printed "trashing") · MCQ "thrashing occurs when": BCS MCQ sheet, paper not shown

Paging: memory divided into fixed-size **frames**, a process into equal **pages**; any page can go in any frame, so no external fragmentation. **Thrashing**: so little RAM per process that it **spends more time swapping pages than executing**; CPU use collapses. Cure: lower the degree of multiprogramming, **working-set** model, page-fault-frequency control, more RAM.



3 LRU page replacement: count the faults

Q6: 3 page frames, initially empty, LRU. Page faults for the reference string 4, 7, 6, 1, 7, 6, 1, 2, 7, 2?

✎ **Asked in:** BPDB AE(CSE) 2021 · shorter string 4, 7, 6, 1, 2, 7, 2: BPDB AE(CSE) 2024

Ref	4	7	6	1	7	6	1	2	7	2
Frames	4	4 7	4 7 6	1 7 6	1 7 6	1 7 6	1 7 6	1 2 6	1 2 7	1 2 7
Fault?	F	F	F	F	hit	hit	hit	F	F	hit

→ **6 page faults** (hit ratio 4/10). At "2": 7 was used longest ago → evicted. Shorter string: 4 7 6 1 2 7 all fault, last 2 hits → also **6**.

Same string, 3 frames	Rule	Victim at "2" (t = 8)	Faults
FIFO	evict the page loaded first	7 (loaded before 6 and 1)	6
LRU	evict the page used longest ago	7 (last used at t = 5)	**6**
Optimal	evict the page needed farthest ahead	6 (never needed again)	**5**

Q7: Explain the LRU and NRU page-replacement algorithms.

✎ **Asked in:** Combined Bank AME/AE(IT) 2024 · LRU with an example: NTRCA question bank · GUI, RAID, API, LRU table: BPDB AE(CSE) 2023

LRU (Least Recently Used): Evict the page **not used for the longest time**; uses the past to predict the future. Implemented by a counter/timestamp per page or a stack of page numbers; costly in hardware, so real systems use approximations (clock / second chance). No Belady's anomaly.

NRU (Not Recently Used): Each page has **R** (referenced) and **M** (modified) bits; R is cleared on every clock tick. Pages fall into 4 classes; evict a random page from the **lowest class**.

Class	R	M	Meaning	Evict?
0	0	0	not referenced, not modified	**first choice**
1	0	1	not referenced, modified (must write back)	second
2	1	0	referenced, clean	third
3	1	1	referenced and modified	last

Exam trap: **FIFO** can show **Belady's anomaly** (more frames → more faults, e.g. 1 2 3 4 1 2 5 1 2 3 4 5: 9 faults with 3 frames, 10 with 4). LRU and Optimal never do. Optimal (evict the page used **farthest in the future**) is the lower bound but needs the future.

4 Paging numericals

Q8: Total number of pages when page size is 4 KB and the address space is 32 bits.

✎ **Asked in:** SGFL AE(IT) 2023

offset = $\log_2 4096$ = **12** bits → page no. = 32 − 12 = 20 bits

→ 2^{20} = **1,048,576 pages**

Q9: 64 MB physical memory, 32-bit virtual address, 4 KB pages: approximate size of the page table?

✎ **Asked in:** 45th BCS ICT 2024

entries = 2^{20} · frames = $2^{26} / 2^{12} = 2^{14}$ → frame no. 14 bits ≈ **2 bytes** per entry

→ $2^{20} \times 2$ B = **2 MB**

State the entry size: with the usual 4-byte entry (frame no. + valid, dirty, protection bits) the answer is $2^{20} \times 4$ B = **4 MB**. Both are accepted if the assumption is written.

Q10: 60 MB of memory is divided into fixed partitions of 10, 12, 18 and 20 MB. Programs of 17, 8, 10.5 and 20 MB occupy the 3rd, 1st, 2nd and 4th partitions. Find the total memory, memory used, memory allocated, memory wasted and the percentage wasted.

✎ **Asked in:** Petrobangla ANE suggestion sheet (no dated paper)

Partition	Program	Wasted
10 MB	8 MB	2 MB
12 MB	10.5 MB	1.5 MB
18 MB	17 MB	1 MB
20 MB	20 MB	0

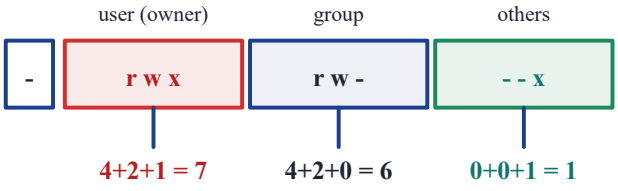
1. total memory = **60 MB**
2. used = 17 + 8 + 10.5 + 20 = **55.5 MB**
3. allocated = all 4 partitions = **60 MB**
4. wasted = 60 − 55.5 = **4.5 MB**
5. 4.5 / 60 × 100 = **7.5 %**

All this waste is **internal** fragmentation (inside allocated partitions). **Page vs block** ✎ **Asked in:** Combined Bank SO(IT) 2020: in an SSD, a **page** (4–16 KB) is the smallest unit read or written; a **block** (e.g. 128–256 pages) is the smallest unit **erased**; SSD → controller → flash chips → blocks → pages.

5 File permissions

Q11: Name the three classes of users who can access a file or directory on Linux.

✎ **Asked in:** BPDB AE(CSE) 2023



r = 4 · w = 2 · x = 1 → rwxrw---x = 761

u = user (owner) · **g** = group · **o** = others (**a** = all three). First character: `-` file, `d` directory, `l` link. On a directory, x = may enter it, r = may list it.

Memory trick: "**Read 4, Write 2, eXecute 1**", add per class: 7 rwx · 6 rw- · 5 r-x · 4 r-- · 0 ---.

Q12: A file's permission is rwxrw---x. After `chmod 701` it becomes ____.
✎ **Asked in:** BPDB AE(CSE) 2021 (printed "rwxrwr--x", an impossible string)

→ **rwx-----x**: 7 = rwx, 0 = ---, 1 = --x. Numeric chmod **replaces** all bits, so the old value does not matter.

6 Task → command (one line each)

Task	Command	Asked in
read, write, execute for all	`chmod 777 apscl.txt` (or `chmod a+rwx`)	✎ APSCL AE(ICT) 2021 · IBBL SO(Network) 2025
show a file's permissions	`ls -l myfile` · `stat myfile`	✎ BTCL JAM 2022 · MGMCL AM(ICT) 2022 · BTCL AME 2023
change owner / group	`chown rahim:staff file` · `chgrp staff file`	✎ BUET M.Sc. · PBL HE 2023
all files incl. hidden	`ls -a` · home dir: `ls -la ~`	✎ Combined Bank ADA 2022 · NESCO AM(ICT) 2021 · Milk Vita AM(CSE) 2023 · MGMCL AM(ICT) 2022
list all files of a folder	`ls folder` · details: `ls -l`	✎ BTCL JAM 2022
current directory	`pwd`	✎ BTCL AME 2023
make directory PSC	`mkdir PSC` (nested: `mkdir -p a/b`)	✎ BPSC SAE 2021 · BTCL JAM 2022
copy a directory with contents	`cp -r dir /home/admin/`	✎ BPSC SAE 2021
move / rename; change dir	`mv a.txt docs/` · `cd /var/log`	✎ NPCBL ET(Soft) 2023
create, then delete a file	`touch apscl.txt` · `rm apscl.txt`	✎ APSCL AE(ICT) 2021 · MGMCL AM(ICT) 2022
delete a directory and its files	`rm -r dir` · only contents: `rm -rf dir/*`	✎ Milk Vita AM(CSE) 2023 · IBBL SO(Network) 2025
create a user	`sudo useradd -m rahim` · `passwd rahim`	✎ PGCB SAE 2021

Exam trap: `rmdir` removes only an **empty** directory; use `rm -r`. `rm -rf dir/*` leaves hidden files (`.x`) behind. `chmod 777` answers the question but is a security risk: say so in one line.

7 Viewing and searching files

Sample: `myfile` holds 20 lines "line 1" … "line 20"; `app.log` holds 4 lines (boot ok · disk error on sda · login ok · net error eth0); `main.c` 3 lines, `util.c` 4, `util.h` 1.

Task	Command → output	Asked in
last 10 lines of a log that keeps growing	`tail -f app.log` (`-n 10` is the default; **-f** follows new lines)	✎ Titas Gas AE(CSE) 2024
last five lines	`tail -n 5 myfile` → line 16 … line 20	✎ Milk Vita AM(CSE) 2023
first 10 / first 7 lines	`head myfile` · `head -n 7 apscl.txt` → line 1 … line 7	✎ BUET M.Sc. · APSCL AE(ICT) 2021
11th to 15th line	`sed -n '11,15p' myfile` → line 11 … line 15 or `head -n 15 myfile | tail -n 5`	✎ BTCL AME 2023
search a word / pattern	`grep -n "error" app.log` → 2:disk error on sda · 4:net error eth0	✎ BUET M.Sc. · MGMCL AM(ICT) 2022
print from a specific line on	`sed -n '/disk/,$p' app.log` → disk error … net error eth0 · by number: `tail -n +2`	✎ NPCBL ET(Soft) 2023
download a file from a URL	`wget https://site/file.zip` · `curl -O URL`	✎ Milk Vita AM(CSE) 2023

Q13: Write a Linux command to display the total number of lines of all .c and .h files in the current directory.

✎ **Asked in:** BPDB AE(CSE) 2023

```
$ cat *.c *.h | wc -l
8
$ wc -l *.c *.h
  3 main.c
  4 util.c
  1 util.h
  8 total
```

Exam trap: `ls *.c *.h | wc -l` prints **3**: it counts **files**, not lines. The question asks for lines: **8**.

Q14: Combine history and grep to find previously used commands.

✎ **Asked in:** BCC AP 2021

`history | grep "ssh"`: **history** prints the numbered command list, the pipe `|` feeds it to **grep**, which keeps only lines containing "ssh". Re-run number 42 with `!42`; search interactively with **Ctrl + R**.

Q15: Shell script that adds the line "This is my file" at the top of every .txt file in the current directory, keeping the rest of the content.

✎ **Asked in:** BPDB AE(CSE) 2021

```
#!/bin/bash
for f in *.txt; do
  sed -i '1i This is my file' "$f"
done
```

x.txt "a" → This is my file / a
y.txt "b c" → This is my file / b / c
Without sed: `{ echo "This is my file"; cat "$f"; } > tmp && mv tmp "$f"`

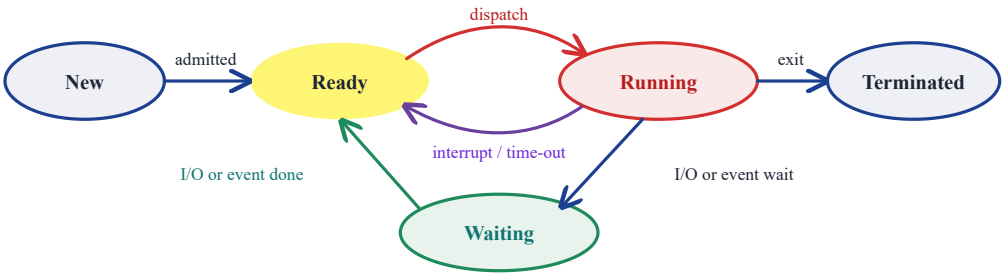
> **Q16:** Go to the home directory, list its files with all details and human-readable sizes, and show only lines 10–15 of the listing, all in one line.
> ✎ **Asked in:** BGDCL AE(CSE) 2021

`cd ~ && ls -lh | sed -n '10,15p'` · **System:** page size `getconf PAGESIZE` (4096 on x86-64) and disk space `df -h` ✎ **Asked in:** NESCO AM(ICT) 2021 · kill a process `kill 1234` / `kill -9 1234` / `pkill name` ✎ **Asked in:** BUET M.Sc. · IP address `ip addr`, partitions `lsblk` / `sudo fdisk -l` ✎ **Asked in:** IBBL SO(Network) 2025 · also ✎ **Asked in:** GTCL AE(CSE) 2022

8 Process and its states

> **Q17:** What is a process? Describe the process states, with the process state transition diagram.
> ✎ **Asked in:** 17th NTRCA ICT 2023 · 18th NTRCA College ICT 2024 · EGCB SAE(ICT) 2023 · BPSC AP 2021 (diagram) · program vs process: NTRCA question bank

A **process** is a **program in execution**: code + program counter + registers + stack + data + heap + OS bookkeeping (PCB). A **program** is passive (a file on disk); a process is active, and one program can run as many processes.



five-state model: only Running → Waiting and Waiting → Ready involve I/O; Waiting never goes straight to Running

State	What the process is doing
New	being created; PCB built, not yet admitted
Ready	in RAM, has everything except the **CPU** (in the ready queue)
Running	instructions executing (one per core at a time)
Waiting (blocked)	waiting for I/O or an event; cannot use the CPU even if free
Terminated	finished; PCB kept until the parent reads the exit status

> **Q18:** What is a PCB? Write the information it keeps about a process (four items).
> ✎ **Asked in:** 17th NTRCA ICT 2023 · RPGCL AM(ICT) 2022 · four process attributes kept for a thread: BPSC AME 2021

- ✓ **Process ID** (PID), parent ID
- ✓ **Process state**
- ✓ **Program counter** + CPU registers
- ✓ CPU-scheduling info (priority, queue pointers)
- ✓ Memory info (page / segment table, limits)
- ✓ Accounting (CPU time used) and I/O status (open files, devices)

PCB: The **Process Control Block** is the kernel's record of one process; the OS saves the CPU state into it at each switch. For a **thread** the kernel keeps a smaller TCB: thread ID, PC, registers, stack pointer, state, priority.

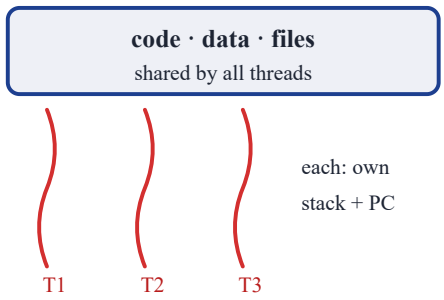
Q19: Explain context switching in an operating system.
✎ **Asked in:** MGMCL AM(ICT) 2022 · BIWTA AE(CSE) 2023 · Combined Bank AME/AE(IT) 2024

Context switch: the CPU moves from process A to B. Steps: interrupt or system call → **save** A's PC and registers in PCB-A → update A's state → scheduler picks B → **load** B's state from PCB-B → resume B. It is **pure overhead** (no useful work), so a very small RR quantum wastes CPU time.

9 Threads vs processes

Q20: Differentiate between a thread and a process. Give two advantages of threads over processes.
✎ **Asked in:** BPSC AME 2021 · BCC Programmer 2019 · Bank AP written (bank name cut off on the sheet)

Process	Thread
program in execution, unit of **resource ownership**	path of execution inside a process, unit of **scheduling**
own address space	**shares** code, data, files of its process
heavy: create / switch costs more	light: own only PC, registers, stack
talk through IPC (pipe, socket)	talk through shared memory
one crashes, others survive	one bad thread can kill the process



Advantages of threads: ① faster to create and switch (no new address space) ② share data without IPC ③ responsiveness: a GUI stays alive while a worker thread blocks ④ use several cores for one task.

10 Multitasking, multithreading, multiprocessing

Q21: What are multitasking and multithreading, and how do they differ? Also: multitasking vs multiprocessing.
✎ **Asked in:** RAKUB ME 2021 · BCC Programmer 2019 · vs multiprocessing: 45th BCS ICT 2024

Term	Meaning	CPUs	Example
Multiprogramming	several jobs in RAM; CPU switches when one waits for I/O	1	batch systems
Multitasking	time-sharing of the CPU among **processes** by time slices	1+	browser + music player
Multithreading	several **threads** of one process run concurrently	1+	Word: typing + spell-check
Multiprocessing	**two or more CPUs** execute truly in parallel	2+	multi-core servers (SMP)

Memory trick: Multi**tasking** = many **tasks** (processes) on time slices · multi**threading** = many threads, one process · multi**processing** = many **processors**. Only multiprocessing needs extra hardware.

11 Synchronization

Q22: Explain race condition in process synchronization.

✎ **Asked in:** 17th NTRCA ICT 2023 · NTRCA question bank

Two processes update shared data at once and the **result depends on the order** of execution. counter = 5; P1 does counter++, P2 counter−−; interleaved load/store can leave **4 or 6** instead of 5. Fix: make the **critical section** mutually exclusive (mutex, semaphore).

Q23: Explain the producer–consumer problem and the synchronization issue to solve.

✎ **Asked in:** Bank AP written (bank name cut off on the sheet)

Producer puts items in a bounded buffer of N slots; consumer takes them. Must not add to a **full** buffer, take from an **empty** one, or touch it together. Semaphores: `empty = N`, `full = 0`, `mutex = 1`.

```
producer: wait(empty); wait(mutex); add item;  signal(mutex); signal(full);
consumer: wait(full);  wait(mutex); take item; signal(mutex); signal(empty);
```

A **semaphore** is an integer changed only by atomic **wait (P)**, which decrements and blocks below 0, and **signal (V)**, which increments and wakes a waiter ✎ **Asked in:** semaphore, readers–writers: NTRCA question bank. Swapping the two waits in the producer can deadlock.

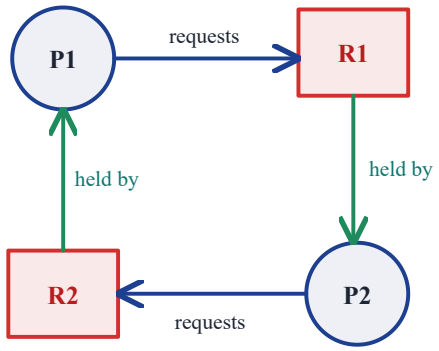
12 Deadlock and its four conditions

Q24: What is deadlock in an OS? What are the four necessary conditions behind it? Draw a diagram of a deadlock situation.

✎ **Asked in:** BPSC AP 2023 · Milk Vita AM(CSE) 2023 · BTCL JAM 2024 · BPSC AP 2022 · 45th BCS CSE 2024 (diagram) · NTRCA question bank

Deadlock: a set of processes in which **each waits for a resource held by another** in the set, so none can ever proceed. All four **Coffman conditions** must hold at once:

1. **Mutual exclusion**: a resource is held by one process at a time
2. **Hold and wait**: a process holds one resource while waiting for another
3. **No preemption**: a resource is released only voluntarily
4. **Circular wait**: P1 → P2 → … → Pn → P1, each waiting for the next



resource-allocation graph: a cycle = circular wait

Memory trick: "**My Hungry Neighbour Cries**": Mutual exclusion, Hold and wait, No preemption, Circular wait. Break **any one** → no deadlock.

Q25: Can a deadlock involve only a single process? Explain.

✎ **Asked in:** Combined Bank Officer(IT) 2024

Not by the definition: **circular wait needs ≥ 2 processes**. In practice one thread can block itself forever by locking a non-recursive mutex it already holds ("self-deadlock"), but that is a bug, not the four-condition deadlock.

Q26: Explain circular-wait deadlock.

✎ **Asked in:** Titas Gas AE(CSE) 2024

P1 holds R1 and wants R2, P2 holds R2 and wants R1: the waits form a cycle. Prevent it by **resource ordering**: number all resources and request them only in increasing order, so a cycle cannot form.

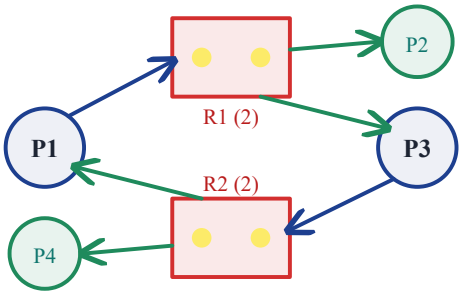
Q27: Resources A and B; processes P1, P2. A free resource is allocated at once, otherwise the requester halts. Sequence: P1 requests A, P2 requests B, P1 requests B, P2 requests A. (i) A is allocated to? (ii) B is allocated to? (iii) Name the situation.

✎ **Asked in:** DPDC JAM(ICT) 2020

→ (i) A → **P1** · (ii) B → **P2** · (iii) P1 halts waiting for B, P2 halts waiting for A: **deadlock** (circular wait)

Q28: Show by example that mutual exclusion, hold and wait, no preemption and circular wait are not sufficient for a deadlock.

✎ **Asked in:** DPDC AM(ICT) 2025



R1 and R2 have **2 instances** each. Cycle P1 → R1 → P3 → R2 → P1 exists, so all four conditions hold. But **P2** holds the other R1 and **P4** the other R2, and they wait for nothing: when P2 finishes, R1 goes to P1, and the cycle breaks. **No deadlock**. With single-instance resources a cycle **is** a deadlock; with several instances it is **necessary but not sufficient**.

Q29: When does deadlock occur and how do you prevent it? Describe the ways of detecting and protecting against deadlock / of handling it.

✎ **Asked in:** 45th BCS ICT 2024 · 38th BCS 2018 · 18th NTRCA College ICT 2024

Method	Idea	How
Prevention	make one condition impossible	share read-only files (ME) · request all at start (H&W) · take resources back (NP) · **ordered requests** (CW)
Avoidance	grant only if the state stays **safe**	**Banker's algorithm** (needs each maximum in advance)
Detection + recovery	let it happen, find it, fix it	wait-for graph cycle / detection algorithm → kill processes or preempt resources (rollback)
Ignore	ostrich algorithm	Windows, Linux for user programs: rare, so reboot

13 Banker's algorithm, worked

Q30: 5 processes P0–P4; resources A = 10, B = 5, C = 7 instances. Snapshot below, Available = (3, 3, 2). Find Need and check whether the system is safe.

✎ **Asked in:** RAKUB ME 2021

P	Alloc	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

1. Need = Max − Allocation
2. Work = 3 3 2 → P1 (1 2 2 ≤ work) runs, work = **5 3 2**
3. P3 (0 1 1) → work = **7 4 3**
4. P4 (4 3 1) → work = **7 4 5**
5. P2 (6 0 0) → work = **10 4 7**
6. P0 (7 4 3) → work = **10 5 7** = total ✓

→ Safe; safe sequence **⟨P1, P3, P4, P2, P0⟩** (⟨P1, P3, P0, P2, P4⟩ is also valid: any order that passes the test scores).

14 The formula question

Q31: A system has P processes, each needing at most m resources, and r resources in total. What condition makes it deadlock-free?

✎ **Asked in:** BPSC AP 2023

Worst case: every process holds **m − 1** and waits for one more. If even then one resource is left over, some process can finish and release all it holds: **P(m − 1) + 1 ≤ r**. Example: 3 processes need 3 each → 3 × 2 + 1 = 7 resources needed.

MCQ, faulty print: "4 processes, 3 resources: maximum that can run without deadlock?" ✎ **Asked in:** 49th BCS ICT 2025 gives no per-process need. Read as **3 instances, each process needs 2**: n(2 − 1) + 1 ≤ 3 → **n = 2**. The printed key (3) does not fit that reading; write the assumption.

Other MCQs asked: Most prominent method to **avoid** deadlock → **Banker's** ✎ **Asked in:** 49th BCS ICT 2025 · resources always preempted from the same process → **starvation** ✎ **Asked in:** BPDB AE(CSE) 2021 · **spooling** (print jobs queued on disk so the CPU need not wait for the printer) is for printer / device management ✎ **Asked in:** 49th BCS ICT 2025 · dining philosophers: 5 philosophers each pick up the left fork and wait for the right → all wait forever ✎ **Asked in:** BPSC AME 2021; fixes: at most 4 at the table, pick up both forks together, odd ones pick left first and even ones right first.

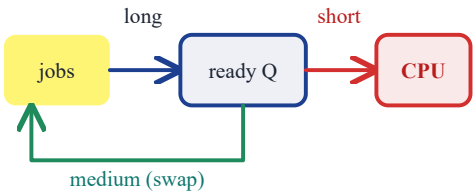
15 Why and how the CPU is scheduled

Q32: What is CPU scheduling? Why is it needed (criteria)? List and briefly define the major types of OS scheduling.

✎ **Asked in:** 17th NTRCA ICT 2023 · types: Combined Bank Officer(IT) 2020

CPU scheduling decides which ready process gets the CPU next. Needed to keep the CPU busy while others wait for I/O and to share it fairly. **Criteria:** maximise **CPU utilisation** and **throughput**; minimise **turnaround**, **waiting** and **response** time.

Scheduler	Decides	Runs
Long-term (job)	which jobs enter RAM; degree of multiprogramming	rarely
Medium-term	swap out / in (suspend)	sometimes
Short-term (CPU)	which ready process runs next	every few ms



Q33: What is the difference between preemptive and non-preemptive scheduling?

✎ **Asked in:** 37th BCS

Non-preemptive	Preemptive
process keeps the CPU until it ends or blocks	OS can **take the CPU away** (timer, higher priority arrives)
FCFS, SJF, non-preemptive priority	RR, SRTF, preemptive priority
simple, low overhead	good response; switch overhead

Q34: Define FCFS, SJF and RR (quantum = 20).

✎ **Asked in:** NU AP 2020 (also: turnaround of FCFS and SJF)

FCFS: serve in arrival order (a queue); long first job → **convoy effect**. **SJF**: run the shortest next burst; **minimum average waiting time**, but bursts must be predicted. **RR**: each process gets at most q = 20 ms, then goes to the back of the queue; q large → FCFS, q tiny → all switching.

Q35: Explain starvation in SJF scheduling and name a starvation-free algorithm.

✎ **Asked in:** RPGCL AM(ICT) 2022 (printed unclearly)

If short jobs keep arriving, a long job is **postponed indefinitely** (starvation). Cure: **aging**, raising priority with waiting time. **Round Robin** (and FCFS) are starvation-free: every process gets the CPU within $(n - 1)q$.

Q36: Define socket, kernel, process, multiprogramming, context switching; explain preemptive priority scheduling with an illustration.

✎ **Asked in:** BIWTA AE(CSE) 2023 · Combined Bank AME/AE(IT) 2024

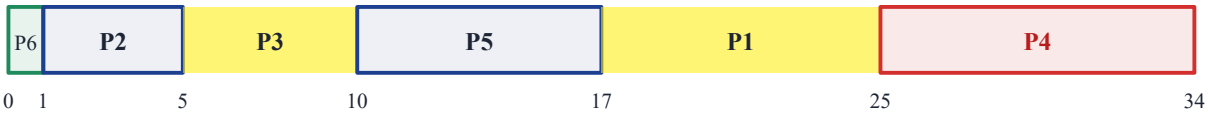
Socket: endpoint (IP + port) of a two-way network connection · **Kernel**: core of the OS, manages CPU, memory, devices (p. 16) · process: p. 8 · context switch: p. 9 · multiprogramming: p. 10. **Preemptive priority**, lower number = higher: P1 (arrives 0, burst 5, pri 3), P2 (1, 3, pri 1), P3 (2, 2, pri 2) →

Gantt: P1 0–1 | P2 1–4 | P3 4–6 | P1 6–10 · TAT 10, 3, 4 (avg 5.67) · WT 5, 0, 2 (avg **2.33**). P2 preempts P1 at t = 1.

16 Six processes: SJF, FCFS, RR

Q37: Six processes with arrival / processing times P1 (0, 8), P2 (0, 4), P3 (0, 5), P4 (1, 9), P5 (1, 7), P6 (0, 1). Find the average turnaround time using SJF. Then average turnaround and waiting time using round robin and FCFS.

✎ **Asked in:** BIWTA AE(CSE) 2023 · Combined Bank AME/AE(IT) 2024 · Teletalk AM(IT) 2023 (table: question-bank sheet, same wording)



SJF: at t = 0 only P1, P2, P3, P6 have arrived → P6 first; then the shortest arrived job

SJF	P1	P2	P3	P4	P5	P6	Average
Completion	25	5	10	34	17	1	
TAT = CT − AT	25	5	10	33	16	1	**90 / 6 = 15**
WT = TAT − BT	17	1	5	24	9	0	**56 / 6 = 9.33**

FCFS (ties by number): P1 0–8 · P2 8–12 · P3 12–17 · P6 17–18 · P4 18–27 · P5 27–34
TAT 8, 12, 17, 26, 33, 18 → avg **19** · WT avg **13.33**

RR, q = 4 (quantum not printed: assumed): P1 P2 P3 P6 P4 P5 P1 P3 P4 P5 P4, ends at 34
TAT 25, 8, 26, 33, 32, 13 → avg **22.83** · WT avg **17.17**

Exam trap: SJF gives the **lowest average WT** (9.33 vs 13.33 vs 17.17); RR is worst on average but best on **response**. Always use **TAT = CT − arrival**, not CT, when arrivals differ (P4, P5 arrive at 1). Here preemptive SJF (SRTF) gives the same chart: no shorter job arrives later.

17 Arrivals spread out: FCFS, SRTF, RR q = 3

Q38: A (0, 10), B (3, 7), C (5, 3), D (8, 5) as (arrival, execution). Average waiting time for FCFS, preemptive SJF and Round Robin with q = 3.
✎ **Asked in:** Petrobangla ANE suggestion sheet (no dated paper)

Algorithm	Gantt chart	WT A B C D	Avg WT
FCFS	A 0–10 · B 10–17 · C 17–20 · D 20–25	0, 7, 12, 12	**7.75**
SRTF	A 0–5 · C 5–8 · A 8–13 · D 13–18 · B 18–25	3, 15, 0, 5	**5.75**
RR q = 3	A 0–3 · B 3–6 · A 6–9 · C 9–12 · B 12–15 · D 15–18 · A 18–21 · B 21–22 · D 22–24 · A 24–25	15, 12, 4, 11	**10.5**

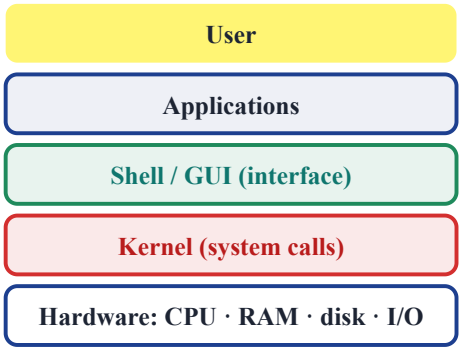
At t = 5 in SRTF, C (3) beats A's remaining 5 → preempt. In RR a newly arrived process joins the queue **before** the one just preempted. **FCFS average waiting**, all arriving at 0 ✎ **Asked in:** RAKUB ME 2021 (bursts not printed; assume 24, 3, 3, 6): WT 0, 24, 27, 30 → **20.25**; run the short ones first and it drops: the convoy effect.

Terms to write first: **CT** completion time · **TAT** = CT − arrival · **WT** = TAT − burst · **Response** = first run − arrival · **Throughput** = processes finished ÷ time (Q37 SJF: 6 / 34) · **CPU utilisation** = busy time ÷ total time

18 What an OS is and does

Q39: What is an operating system? Why do we need it? What is its relationship with the hardware? Main components and functions.

✎ **Asked in:** 38th BCS 2018 · BCC Programmer 2019 · ICT Ministry AP 2017 · DOS, functions and types: 18th NTRCA School 2024



An **operating system** is system software that **manages hardware** and **provides services** to programs, and an interface to users (Windows, Linux, Android). Relationship: hardware gives raw resources; the OS is the **only** layer that drives them directly (drivers, interrupts) and offers them to programs through **system calls**, so applications never touch the hardware.

Functions / components: process management · memory management · file management · I/O and device management · secondary-storage management · security and protection · networking · user interface (CLI / GUI).

DOS (MS-DOS): single-user, single-task, command-line OS. **Types:** batch · multiprogramming · time-sharing · multiprocessing · distributed · network · **real-time** · mobile · embedded.

Q40: Key differences between the shell and the kernel.

✎ **Asked in:** Petrobangla AM(CSE) 2024

Shell	Kernel
interface, reads commands	core, does the work
user mode, replaceable (bash, zsh)	kernel mode, always in RAM
calls the kernel	talks to the hardware

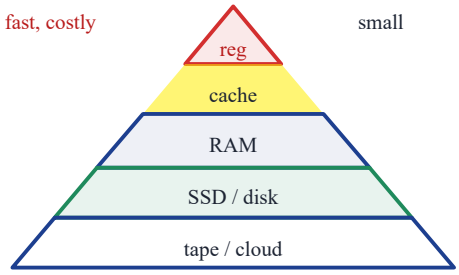
Q41: Difference between micro kernel and macro kernel ("macro" = monolithic, the standard term).

✎ **Asked in:** BB AME 2019 · BPSC AP 2023

Monolithic	Microkernel
all services in kernel space	only IPC, scheduling, basic memory
fast (function calls)	slower (message passing)
one bug crashes all	drivers in user space: robust
Linux, Unix	QNX, Minix, Mach

Real-time system ✎ **Asked in:** 17th NTRCA ICT 2023: correctness depends on the result **and on meeting a deadline**. **Hard** RT: a missed deadline is a failure (airbag, pacemaker, missile control); **soft** RT: quality drops (video streaming). RTOS: VxWorks, FreeRTOS.

Windows NT I/O manager ✎ **Asked in:** BB AME 2019: four sub-components: **cache manager**, **file-system drivers**, **network drivers**, **hardware device drivers**.



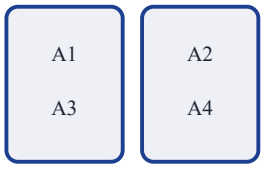
memory hierarchy ✎ **Asked in:** 17th NTRCA CSE 2023

19 RAID

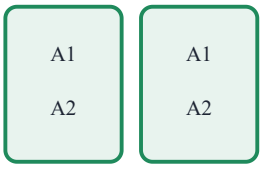
Q42: What is RAID technology? Describe the RAID levels and their advantages. Why is it important for servers in a data centre?

✎ **Asked in:** MoL AME 2023 · BITAC AME(ICT) 2021 · Combined Bank AME/AHE 2023 · purpose of RAID: PBL HE 2023

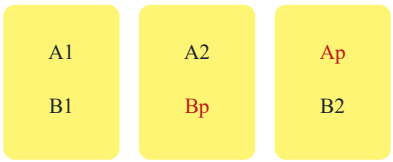
RAID (Redundant Array of Independent / Inexpensive Disks) combines several disks into one logical unit for **speed** (striping), **fault tolerance** (mirroring / parity) or both. Servers need it because a disk failure must not stop the service or lose data, and many disks in parallel give more I/O.



RAID 0: striping



RAID 1: mirroring



RAID 5: rotating parity

Level	Min disks	Capacity (n disks of S)	Survives	Use
RAID 0	2	n × S (4 × 2 TB = 8 TB)	0 failures	speed, scratch data
RAID 1	2	S (mirror pair: 2 TB)	1	OS disk, small DB
RAID 5	3	(n − 1) × S = 6 TB	1	file / read-heavy servers
RAID 6	4	(n − 2) × S = 4 TB	2	large archives
RAID 10	4	n × S / 2 = 4 TB	1 per mirror	**databases**, banking

Q43: Which RAID level is best and why? RAID for a database; RAID for an organisation with huge data processing.

✎ **Asked in:** Sonali Bank ADBA 2024 · RAKUB ME 2021

No single best. For a busy **database**: **RAID 10**, fast random writes and quick rebuilds. Huge capacity, mostly reads: RAID 6 (survives 2 failures during a long rebuild).

Q44: How do you recover when a drive fails in a RAID array?

✎ **Asked in:** Petrobangla AM(CSE) 2024

① confirm the failed disk from the controller / `mdadm --detail` alert; array runs **degraded** ② back up critical data now ③ hot-swap the disk (or let the hot spare take over) ④ **rebuild** (`mdadm --add`), watch progress ⑤ verify; restore from backup if a second disk fails.

Exam trap: **RAID is not a backup**: a deleted file or ransomware is deleted on every mirror at once. RAID 0 has **no** redundancy despite the name.

20 Disk and I/O

Q45: Write the difference between blocking and buffering in an OS.

✎ **Asked in:** 17th NTRCA CSE 2023

Blocking groups several logical records into one physical block (blocking factor = records per block) to cut the number of I/Os. **Buffering** holds data in a memory area while it moves between devices of different speed, so I/O overlaps with computing.

Q46: What is access time and transfer time?

✎ **Asked in:** BTV AP 2019

Access time = seek time + rotational latency (+ transfer): the delay before data starts to flow. **Transfer time** = bytes ÷ transfer rate: moving the data itself. Average latency = half a revolution (7200 rpm → 4.17 ms).

Q47: Disk pack: 16 surfaces, 128 tracks per surface, 256 sectors per track, 512 bytes per sector. (i) capacity (ii) bits to address a sector (iii) formatted space with 32 bytes overhead per sector.

✎ **Asked in:** Petrobangla ANE suggestion sheet (no dated paper)

→ (i) $2^4 \times 2^7 \times 2^8 \times 2^9 = 2^{28}$ B = **256 MB** · (ii) sectors = 2^{19} → **19 bits** · (iii) $2^{19} \times (512 - 32)$ = **240 MB**

★ Night before the exam

1. pages = 2^(address bits) ÷ page size
2. page table = entries × entry size (state it)
3. LRU: evict the one used longest ago
4. Paging → internal, segments → external
5. 4 conditions: ME, H&W, no preemption, CW
6. deadlock-free if $P(m - 1) + 1 \leq r$
7. Need = Max − Alloc; Work grows by Alloc
8. TAT = CT − AT · WT = TAT − BT
9. SJF = min avg WT; RR = no starvation
10. chmod: r 4, w 2, x 1 per u g o
11. lines: `cat *.c | wc -l`, not `ls`
12. RAID 5 = (n − 1)S, RAID 10 = nS/2

✓ Practice MCQs (asked ones first)

1. A zombie process in Unix is one: ✎ 49th BCS ICT 2025

(a) stuck in an infinite loop (b) whose parent has not read its exit status
(c) waiting for I/O (d) in a deadlock

2. A process control block includes: ✎ 49th BCS ICT 2025

(a) process ID (b) process state
(c) program counter (d) all of these

3. Most prominent method to avoid deadlock: ✎ 49th BCS ICT 2025

(a) Banker's algorithm (b) elevator algorithm
(c) Karn's algorithm (d) none

4. Resources always preempted from the same process cause: ✎ BPDB AE(CSE) 2021

(a) deadlock (b) thrashing
(c) starvation (d) aging

5. rwxrw---x after `chmod 701`: ✎ BPDB AE(CSE) 2021

(a) rwxrw---x (b) rwxr----x
(c) rw------x (d) rwx-----x

6. Spooling is mainly used for: ✎ 49th BCS ICT 2025

(a) printer management (b) CPU scheduling
(c) deadlock detection (d) memory allocation

7. Thrashing occurs when processes: ✎ BCS MCQ sheet

(a) wait for I/O (b) swap pages more than they execute
(c) are fragmented (d) under-use the CPU

8. Pages for a 32-bit address and 4 KB pages: ✎ SGFL AE(IT) 2023

(a) 2^{12} (b) 2^{32}
(c) 2^{20} (d) 2^{10}

9. Not a necessary condition for deadlock:

(a) mutual exclusion (b) circular wait
(c) hold and wait (d) preemption

10. Gives the minimum average waiting time:

(a) SJF (b) FCFS
(c) RR (d) priority

Answer key → 1-b · 2-d · 3-a · 4-c · 5-d · 6-a · 7-b · 8-c · 9-d · 10-a

Written question: For the six processes of Q37 (p. 14), draw the SJF Gantt chart and find the average turnaround and waiting time. 10 marks

✎ **Asked in:** BIWTA AE(CSE) 2023 · Combined Bank AME/AE(IT) 2024 · Teletalk AM(IT) 2023

1. At t = 0 the arrived jobs are P1, P2, P3, P6 → shortest P6 runs 0–1 2
2. Then always the shortest arrived: P2 1–5, P3 5–10, P5 10–17, P1 17–25, P4 25–34 3
3. TAT = CT − AT: 25, 5, 10, 33, 16, 1 → average **15** 3
4. WT = TAT − BT: 17, 1, 5, 24, 9, 0 → average **9.33** 2

Before the exam ✓

- ✓ LRU trace table + fault count
- ✓ 4 deadlock conditions + Banker's
- ✓ Gantt chart with arrivals
- ✓ the Linux commands of p. 5–8